

RPA als Steuerungs- & Risikohebel

Plattformüberblick, Attended vs. Unattended –
verantwortete Automation unter Unsicherheit.

Lehrskript – Tag 11 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Robotic Process Automation (RPA) ist heute eines der häufigsten Werkzeuge zur Effizienzsteigerung in Verwaltungs- und Servicebereichen – und gleichzeitig eine der häufigsten Quellen für Schatten-IT, Compliance-Vorfälle und unkontrollierte Skalierung. Heute lernen Sie, RPA als *Steuerungs- und Risikohebel* zu denken: einzuordnen, was sich für Automation eignet, die richtige Form (attended oder unattended) bewusst zu wählen und Entscheidungen unter Zielkonflikten verantwortungsfähig zu treffen.

L1 – Wissen (Reproduktion und Einordnung)

- RPA-Eignungskriterien benennen: regelbasiert, stabiler Prozess, klarer Input/Output, hohes Volumen, geringe Ausnahmen.
- Plattformüberblick einordnen: Studio/Designer, Orchestrierung, Queues, Credentials, Monitoring – toolneutral.
- Unterschied attended vs. unattended verstehen: Betriebsmodell, Skalierungs- und Governance-Konsequenzen.
- Nutzen und Risiken von RPA aus Senior-Brille einordnen: Schatten-IT, Credential-Risiko, fragile UI-Automation, Audit Trail.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Aus einer Use-Case-Liste die Eignung mit Scoring 1–5 bewerten und priorisieren.
- Eine Stufenstrategie attended → unattended entwerfen, inkl. Readiness-KPIs.
- Einen Bot-Flow mit Validierung, Verarbeitung, Logging und Exception Queue konzipieren.
- Audit-Trail-Minimum und KPI-Set (Produktivität, Qualität, Risiko) auf einen Fall anwenden.

L3 – Management (Entscheidung und Verantwortung)

- Ein RPA-Portfolio mit Kriterien Value/Risk/Effort und Kill Criteria steuern.
- Governance leicht, aber verbindlich verankern (Bot Owner, Process Owner, Security, Operations).
- Security- und Compliance-Guardrails (Least Privilege, SoD, Vault) als Pflichtprogramm setzen.
- Entscheidungen unter Unsicherheit (welche Use Cases zuerst, Unattended-Grenze) mit Frühwarnsignalen verankern.

Roter Faden

RPA ist kein Klick-Roboter, sondern ein *privilegierter Mitarbeiter*, der im Sekundentakt entscheidet und bucht. Wer ihn ohne Governance und ohne Audit Trail laufen lässt, gibt einer Software die Vollmachten eines Sachbearbeiters – ohne den menschlichen Bremshebel. Verantwortbare Automation heißt: Eignung prüfen, kontrolliert scheitern, transparent skalieren. (vgl. van der Aalst et al., 2018; Willcocks et al., 2017)

1. Einstieg: Warum RPA mehr ist als Klick-Magie

In vielen Organisationen begann RPA als Selbsthilfe-Werkzeug einzelner Fachbereiche: “Wir haben hier dreihundert Rechnungen pro Tag und drei Mitarbeitende, die nichts anderes tun als Kopieren aus E-Mail in ERP. Können wir das automatisieren?” Die Antwort eines RPA-Tools lautet typischerweise: “Ja, in zwei Wochen.” Was als pragmatische Antwort beginnt, kann zu einer schwer beherrschbaren Plattform mit hunderten Bots, instabilen Portalen und unklaren Verantwortlichkeiten heranwachsen. (Willcocks et al., 2017)

Die Frage ist daher nicht: *Können wir RPA?* Sondern: *Können wir RPA verantworten?*

1.1 Wofür RPA geeignet ist – und wofür nicht

RPA arbeitet typischerweise an der Benutzeroberfläche oder über strukturierte Schnittstellen. Es kopiert, klickt, liest – so wie ein Mensch es täte, nur schneller und fehlerärmer (in stabilen Fällen). Die typischen Eignungskriterien: (Scheppeler & Weber, 2020)

- **Regelbasiert.** Es gibt klare Wenn-dann-Regeln, keine Ermessensentscheidungen.
- **Stabile UI / stabile Schnittstellen.** Das Zielsystem ändert sich selten. Jede UI-Änderung kann den Bot brechen.
- **Klarer Input und Output.** Der Bot weiß, was hereinkommt, und was er produzieren soll.
- **Hohes Volumen.** Lohnt sich erst ab einer relevanten Stückzahl. Drei Vorgänge pro Monat sind kein RPA-Kandidat.
- **Wenige Ausnahmen.** Wenn jeder fünfte Fall anders ist, wird der Bot zur Sammelstelle für Sonderlocken.

Weniger geeignet sind Prozesse mit häufigen UI-Änderungen, vielen Sonderfällen, unklaren Regeln und hohem Interpretationsanteil. Hier hilft RPA allenfalls in Kombination mit AI-Komponenten (Document Understanding, Klassifikation) – aber das ist eine andere Disziplin, mit anderen Risiken.

1.2 RPA im BPM-Lifecycle

RPA steht nicht neben BPM, sondern *in* ihm. Im klassischen Lifecycle (Discovery – Analyse – Design – Automation – Monitoring – Optimization) ist RPA eine Automatisierungs-Option neben Workflow-Engines, ERP-Custom-Code oder API-Integration. Die Wahl hängt vom Reifegrad des Prozesses ab: (Dumas et al., 2018, Kap. 10)

- Unausgereifter, dokumentationsarmer Prozess → erst klären, dann automatisieren.
- Stabiler Prozess, stabile Systeme, API verfügbar → eher API-Integration.
- Stabiler Prozess, Legacy-System ohne API, hohes Volumen → klassischer RPA-Fall.

1.3 Die Versuchung des schnellen Erfolgs

RPA hat eine besondere Versuchung: *schnelle Erfolge*. Ein Pilot ist in zwei bis vier Wochen lauffähig, der ROI scheint klar, das Management ist beeindruckt. Das ist der Punkt, an dem Disziplin am wichtigsten ist – denn der zweite, dritte und zehnte Bot bringt die Skalierungsprobleme: (Smeets et al., 2019)

- **Schatten-IT.** Fachbereiche bauen ohne IT-Beteiligung. Niemand kennt den Bestand, niemand wartet, niemand hat Backups.
- **Fragile UI-Automation.** Ein UI-Update legt zehn Bots gleichzeitig lahm.

- **Credential-Risiko.** Bots brauchen Zugänge. Wenn die nicht zentral verwaltet sind, entstehen sicherheitsrelevante Lücken.
- **Fehlbuchungen im großen Maßstab.** Ein Bot, der einmal falsch programmiert ist, macht denselben Fehler tausendmal – in einer Geschwindigkeit, in der kein Mensch eingreifen kann.

Eselsbrücke: R-S-V-A

Vier Eignungskriterien für RPA, in Reihenfolge prüfen:

Regelbasiert – klare Wenn-dann-Logik?

Stabil – ändert sich UI/Prozess selten?

Volumen – relevante Stückzahl pro Monat?

Ausnahmen – niedrige Quote (idealerweise <10%)?

Wenn auch nur eine Antwort “nein” ist, lohnt sich die Eignung noch nicht.

2. Plattformüberblick: Studio, Orchestrierung, Run

RPA-Plattformen haben unterschiedliche Hersteller, aber sehr ähnliche Architekturen. Wer eine Plattform versteht, versteht sie alle – die Begriffe variieren, die Konzepte sind nahezu gleich.

(Lacity & Willcocks, 2018)

2.1 Studio / Designer: das Werkzeug zum Bauen

Im **Studio** (oder Designer) wird die Bot-Logik gebaut. Typische Bausteine: *Workflows* (eine Folge von Schritten), *Activities* oder *Actions* (einzelne Tätigkeiten wie “E-Mail lesen”, “Excel öffnen”, “Daten in Maske eintragen”), *Variablen*, *Bedingungen*, *Schleifen*. Wer schon einmal Workflow-Tools wie Camunda oder Power Automate genutzt hat, findet sich schnell zurecht.

Wichtige Disziplinen im Studio:

- **Naming Convention.** Activities, Workflows und Variablen folgen klaren Benennungsregeln. Sonst wird ein Bot mit 200 Schritten unwartbar.
- **Modularisierung.** Wiederkehrende Bausteine als Sub-Workflows. Wer alle Aktivitäten in einen Workflow packt, baut den klassischen “Spaghetti-Bot”.
- **Fehlerbehandlung.** Try/Catch um kritische Schritte, mit definierter Fehlerklasse.
- **Konfiguration extern.** Hostnames, Pfade, Schwellenwerte nicht im Code, sondern in einer Konfigurationsdatei oder im Orchestrator.

2.2 Orchestrierung: das Kontrollzentrum

Der **Orchestrator** (oder Control Room, Management Console) ist das Herz der Plattform. Er übernimmt: (Smeets et al., 2019)

- **Planung.** Welcher Bot läuft wann, auf welcher Maschine, mit welchem Zeitplan?
- **Queues.** Eingangs- und Ausgangswarteschlangen mit Items, Status, Priorität.
- **Credentials.** Zentral verwaltete Zugangsdaten. Bots greifen über Tokens darauf zu, nie über fest codierte Passwörter.
- **Monitoring.** Status der Bots, Fehlerraten, Durchsatz, Alerts.
- **Logs.** Was hat der Bot wann getan, mit welchem Input, mit welchem Output, mit welchem Fehler?

2.3 Run-Umgebung: wo der Bot lebt

Der eigentliche Ausführungsknoten – in attended Fällen am Mitarbeiterarbeitsplatz, in unattended Fällen auf einer Server- oder Cloud-VM. Drei Aspekte: (Scheppler & Weber, 2020)

- **Isolierung.** Jeder Bot läuft in einer isolierten Umgebung, damit ein abgestürzter Bot nicht andere mitreißt.
- **Ressourcen.** CPU, RAM, ggf. Bildschirmauflösung (wenn UI-Automation). Spielt mehr Bot-Volumen gegen Hardware-Kosten aus.
- **Verfügbarkeit.** Was passiert, wenn die Maschine ausfällt? Failover, Standby, oder akzeptiertes Risiko?

2.4 Low-Code vs. Enterprise

Auf dem Markt gibt es zwei Pole: Low-Code-/No-Code-Plattformen (oft auch Citizen Developer ansprechend) und Enterprise-Plattformen mit umfassender Governance. Beide haben ihre Berechtigung – aber sie skalieren unterschiedlich:

- **Low-Code.** Schnell, niedrige Einstiegshürde, gut für Pilotierung und kleinere Use Cases. Risiko: ohne Standards entsteht Wildwuchs.
- **Enterprise.** Höhere Einstiegshürde, dafür Governance, Audit, Skalierung mitgedacht. Risiko: bürokratisch, langsam, ohne Buy-in nicht akzeptiert.

3. Attended vs. Unattended: zwei Welten, eine Entscheidung

Die wichtigste strategische Entscheidung beim RPA-Einsatz ist die Wahl zwischen *attended* und *unattended* – oder einer abgestuften Kombination. Sie betrifft Betriebsmodell, Governance, Skalierbarkeit und Risiko.

3.1 Attended: Assistent am Arbeitsplatz

Ein **attended Bot** läuft am Arbeitsplatz eines Mitarbeitenden. Der Mitarbeitende triggert ihn (per Klick oder Hotkey), überwacht ihn, greift bei Problemen ein. Typische Anwendungen: Teilschritt-Assistenz (“Bitte fülle diese fünf Felder im ERP automatisch aus”), Datenkonsolidierung, repetitive Klick-Strecken.

- **Vorteile.** Geringeres Betriebsrisiko, niedrige Governance-Hürde, schnelle Lernkurve, Akzeptanz im Team (“der Bot hilft mir, ersetzt mich nicht”).
- **Nachteile.** Begrenzte Skalierung, abhängig von Anwesenheit, schwer auditierbar (es gibt nicht immer einen zentralen Log-Pfad).

3.2 Unattended: serverseitig, vollautomatisch

Ein **unattended Bot** läuft serverseitig (oder in der Cloud), ohne menschliche Anwesenheit. Er wird über Zeitpläne oder Queues angestoßen, arbeitet im Schichtbetrieb (auch 24/7), und meldet Ergebnisse über das Orchestrator-System zurück. (Lacity & Willcocks, 2018)

- **Vorteile.** Hohe Skalierung, kontinuierlicher Betrieb, klar auditierbar (alle Aktionen im zentralen Log), entkoppelt vom Arbeitsplatz.
- **Nachteile.** Höhere Anforderungen an Governance, Security, Exception Handling, Monitoring. Höhere initiale Kosten.

3.3 Hybride und Stufenstrategien

In der Praxis ist die richtige Antwort selten “alles attended” oder “alles unattended”, sondern eine *Stufenstrategie*: (Smeets et al., 2019)

- **Stufe 1.** Attended-Pilot in einem Bereich. Lernkurve aufbauen, Akzeptanz testen, erste Stabilität messen.
- **Stufe 2.** Wenn KPIs (Stabilität, Ausnahmequote) erreicht sind: Übergang in unattended für skalierbare Use Cases, parallel attended für interaktive Hilfen.
- **Stufe 3.** Wenn die Plattform reif ist: Portfolio-Steuerung, CoE-Strukturen, Self-Service für Fachbereiche unter klaren Leitplanken.

Stufenentscheidung – die wichtigste Frage

“Wie können wir *attended* starten, dass uns der spätere Wechsel zu *unattended* nicht noch einmal alles neu kostet?” Diese Frage am Anfang spart Monate Re-Engineering. Standards für Logging, Konfiguration und Exception Handling müssen *attended* schon so sein, wie sie *unattended* sein werden.

3.4 Entscheidungsfaktoren

Faktor	Attended	Unattended
Prozessstabilität	mittel reicht	hoch nötig
Volumen	klein bis mittel	mittel bis sehr hoch
Verfügbarkeitsanspruch	Bürozeiten	24/7 möglich
Audit-Relevanz	gering bis mittel	hoch (zentrale Logs)
Governance-Reife	niedrig reicht	muss vorhanden sein
Akzeptanz im Team	oft hoch	oft Skepsis
Skalierung	begrenzt	sehr gut

Die Faustregel: *Attended* startet schneller, *unattended* skaliert besser. Wer beides will, plant von Anfang an die Stufenstrategie ein.

4. Nutzen und Risiken aus Senior-Brille

RPA-Initiativen scheitern selten an der Technik. Sie scheitern an unterschätzten Nebenwirkungen. Wer RPA verantworten will, muss Nutzen und Risiken bewusst abwägen.

4.1 Der Nutzen – und seine Grenzen

- **Zeitgewinn.** Sachbearbeitende verlagern Zeit von repetitiver Erfassung zu Bearbeitungs- und Entscheidungsaufgaben. Achtung: Zeit *verlagert* sich, sie wird nicht von selbst frei – ohne Rollen-Anpassung wird sie “versickert”.
- **Fehlerreduktion.** Bots tippen nicht falsch, sie vergessen kein Pflichtfeld. Vorausgesetzt, sie wurden korrekt gebaut.
- **SLA-Stabilität.** Durchsatz wird vorhersagbar, Spitzen werden ausgeglichen.
- **Audit Trail.** Wenn richtig gebaut, dokumentiert der Bot lückenlos, was er getan hat – besser als jeder manuelle Vorgang.

4.2 Die Risiken – und ihre wirksamen Gegenmittel

- **Schatten-IT.** Bots entstehen ohne IT-Beteiligung, ohne Backup, ohne Wartungsplan. Gegenmittel: Intake-Funnel mit Pflicht-Anmeldung, Inventur, Sunset-Plan.
- **Credential-Risiko.** Bots brauchen Zugänge. Ohne Vault landen Passwörter in Skripten oder unverschlüsselten Konfigurationen. Gegenmittel: zentraler Credential Vault, getrennte Bot-Accounts, Least Privilege.
- **Fragile UI-Automation.** Eine kleine UI-Änderung legt einen Bot lahm. Gegenmittel: stabile Selektoren, Monitoring auf UI-Änderungen, Test-Bots mit Smoke-Tests vor Releases der Zielsysteme.
- **Fehlbuchungen.** Ein Programmierfehler erzeugt tausend falsche Buchungen, bevor jemand es bemerkt. Gegenmittel: Plausibilitätsgrenzen, Schwellenwerte, manuelle Freigabe ab Betrag X, Sampling-Kontrollen.

4.3 Risiko-Heuristik: wo droht der größte Schaden?

Eine Senior-Frage vor jedem Bot: “Was passiert, wenn dieser Bot zehntausend Mal das Falsche tut?” Drei Schadensdimensionen:

- **Finanziell.** Fehlbuchungen, Doppelzahlungen, falsche Gutschriften.
- **Compliance.** Verstöße gegen Datenschutz, Steuerrecht, regulatorische Vorgaben.
- **Reputation.** Kunden bemerken Fehler, Medien greifen es auf, Vertrauen erodiert.

Die Antwort darauf ist nicht “vorsichtiger programmieren”, sondern *Guardrails einbauen*: harte Limits, Plausibilitätsgrenzen, manuelle Freigaben bei sensiblen Operationen, kontinuierliches Monitoring. (Syed et al., 2020)

5. Entscheidung unter Unsicherheit – Daniel Kahneman trifft RPA

Die meisten RPA-Entscheidungen werden unter Unsicherheit getroffen: das genaue Volumen ist nicht bekannt, die Stabilität des Portals ist unklar, die zukünftige UI-Änderungsrate ist Spekulation. Hier hilft die Heuristik aus der Entscheidungsforschung. (Kahneman, 2011)

5.1 System 1 vs. System 2

Daniel Kahneman unterscheidet zwei Denksysteme: *System 1* (schnell, intuitiv, mustergeleitet) und *System 2* (langsam, analytisch, anstrengend). In RPA-Entscheidungen ist System 1 die Quelle der typischen Fehler:

- “Wir haben das doch schon einmal automatisiert, das geht hier auch.” (Verfügbarkeits-Heuristik)
- “Der Use Case sieht einfach aus, wir starten unattended.” (Optimismus-Bias)
- “Wenn 200 Bots laufen, müssen ja auch 2.000 laufen.” (Linearitäts-Annahme)

System-2-Disziplin bedeutet: Annahmen explizit machen, Frühwarnsignale definieren, schlechte Alternativen ausschließen, bewusst Schritt für Schritt skalieren.

5.2 Vier Disziplinen verantworteter Entscheidung

1. **Annahmen sichtbar machen.** “Wir nehmen an, dass das Portal in den nächsten 6 Monaten nicht geändert wird” – explizit, mit Verfallsdatum.

2. **Verworfenne Alternativen dokumentieren.** Was haben wir nicht gewählt, und warum nicht?
Eine Entscheidung ohne verworfene Alternative ist meist eine Behauptung.
3. **Frühwarnsignale definieren.** Welche Messgröße sagt uns, dass wir nachsteuern müssen?
Ausnahmequote, UI-Änderungsrate, Fehlbuchungsrate.
4. **Kill Criteria akzeptieren.** Ab welchem Punkt stoppen wir den Bot? Nicht “nie” – sondern bei klar definierten Schwellen.

Eselsbrücke: A-V-F-K

Vier Disziplinen bei jeder RPA-Entscheidung unter Unsicherheit:

Annahmen sichtbar (mit Verfallsdatum)

Verworfenne Alternativen begründen

Frühwarnsignale benennen

Kill Criteria vereinbaren

“A-V-F-K” verhindert, dass aus einer Wette eine Behauptung wird.

6. Fallstudie: NordTrade GmbH – Rechnungsverarbeitung

NordTrade GmbH ist ein mittelgroßer Großhändler. Im Buchhaltungsbereich kommen täglich rund 250 Eingangsrechnungen aus E-Mail oder PDF-Portalen herein. Die manuelle Erfassung erzeugt Fehler und Rückfragen. Der CFO will 30 % Zeitersparnis in 8 Wochen. Der Auditor fordert Nachvollziehbarkeit.

Restriktionen: 8 Wochen, Budget 80 k EUR, die UI eines Portals ändert sich gelegentlich, 12 % Ausnahmen (fehlende Daten), Zielkonflikt Speed vs. Auditfähigkeit.

6.1 Automationsform: attended starten, unattended planen

Die gemeinsame Empfehlung von CFO und Compliance: *attended starten*. Begründung:

- Das Portal ist nicht vollständig stabil – ein attended Bot ermöglicht es, UI-Fehler in den ersten Wochen zu beobachten und zu beheben, ohne dass Buchungen aus dem Ruder laufen.
- Der Lernfortschritt des Teams ist höher: Mitarbeitende sehen, was der Bot tut, gewinnen Vertrauen, melden Verbesserungen zurück.
- Nach 6–10 Wochen, bei stabilen KPIs (Exception Rate $\leq 10\%$, UI-Änderungsrate niedrig, Fehlbuchungsrate $< 0,1\%$), Übergang in unattended.

Der Übergang ist nicht “eine Entscheidung später”, sondern bereits jetzt eingeplant – inkl. Logging-Standards, Konfigurations-Trennung und Exception-Code-Schema.

6.2 Bot-Flow

1. **Eingang.** Rechnung empfangen (E-Mail oder Portal-Download).
2. **Daten extrahieren.** Invoice-Nr, Betrag, Lieferant, Datum, ggf. Kostenstelle.
3. **Validierung.** Pflichtfelder, Plausibilität (Betrag > 0 , Datum aktuell, Lieferant bekannt).
4. **ERP-Maske öffnen.** Im SAP/ERP die Buchungsmaske aufrufen.
5. **Daten eintragen.** Felder befüllen, Konsistenzprüfung.
6. **Buchung vorbereiten.** Buchungssatz erzeugen, aber *nicht final speichern*, wenn Betrag über Freigabeschwelle.
7. **Rückmeldung.** Status an Buchhaltung (E-Mail, Ticket, Statusfeld).

8. **Log schreiben.** Audit-Trail-Eintrag mit allen relevanten Feldern.
9. **Ende oder Exception Queue.** Bei Fehler → Exception Queue mit Code und Eskalations-Information.

6.3 Exception-Kategorien und Owner

Code	Beschreibung	Owner / SLA
MissingCostCenter	Kostenstelle fehlt im Rechnungsdokument	Buchhaltung / 4 h
UnknownVendor	Lieferant nicht im Stammdatenregister	Stammdaten-Team / 24 h
DuplicateInvoice	Rechnungsnummer bereits gebucht	Buchhaltung / 2 h
PortalUnreachable	Portal nicht erreichbar	IT / 1 h
ValidationFail	Pflichtfeldverletzung	Buchhaltung / 4 h
ApprovalRequired	Betrag über Freigabeschwelle (z. B. 10.000 EUR)	Vorgesetzter / 1 Werktag

6.4 Audit Trail – Mindestfelder

Jeder Bot-Schritt erzeugt einen Audit-Eintrag mit folgenden Pflichtfeldern:

- Bot-ID + Version (welcher Bot in welcher Version lief)
- Zeitstempel (mit Zeitzone)
- Quelle (Mail-ID oder Portal-Download-Referenz)
- Rechnungs-ID (Case-Korrelation)
- Felder vor / nach (was wurde gelesen, was wurde geschrieben)
- Ergebnisstatus (erfolgreich, in Queue, Exception)
- Exception-Code (falls relevant)
- Manuelle Übernahme (ja/nein, mit Bearbeiter-ID)

6.5 KPI-Set für 8 Wochen

- **Durchsatz pro Tag.** Ziel +30 % gegenüber Baseline.
- **Fehlerquote.** Ziel –50 % gegenüber Baseline.
- **Exception Rate.** Ziel $\leq 10\%$ nach Stabilisierung (Woche 4 ff.).
- **Rollback-/Correction-Quote.** Wieviele Buchungen mussten manuell korrigiert werden? Ziel $< 0,5\%$.

6.6 Frühwarnsignale für den Übergang attended → unattended

- Exception Rate steigt über 15 % → Übergang verzögern.
- Portal-UI-Änderungen > 1 pro Monat → Stabilität testen.
- Korrekturen $> 1\%$ der Buchungen → Validierung verschärfen.
- Auditor meldet Lücken im Audit Trail → Logs erweitern, dann erst weiter.

7. Coaching: RPA als verantwortete Produktivität

Der Sprung von “RPA als Klick-Magie” zu “RPA als verantwortete Produktivität” ist primär ein Denksprung, nicht ein Tooling-Sprung.

7.1 Der Automation Candidate Funnel

Wer RPA verantwortet, baut keine einzelnen Bots, sondern eine Pipeline. Vier Stufen:

1. **Ideen.** Vorschläge aus den Fachbereichen, ungefiltert. “Wir haben hier auch was Repetitives.”
2. **Eignungs-Check.** Bewertung gegen Eignungskriterien (regelbasiert, stabil, Volumen, Ausnahmen). Mindestens 50 % der Ideen scheiden hier aus.
3. **PoC / Pilot.** Kleiner, kontrollierter Test. Verifikation der Eignung in der Realität.
4. **Betrieb.** Nur Use Cases, die PoC bestanden haben und ein klares Operating-Modell haben, gehen in Produktion.

7.2 Ausnahmebehandlung als Kern

In jedem RPA-Projekt gilt: *die Ausnahmebehandlung entscheidet über ROI*. Wenn 12 % der Vorgänge Ausnahmen sind und jede Ausnahme manuelle Bearbeitung kostet, kippt die Wirtschaftlichkeit schnell. Drei Disziplinen:

- **Klassifizieren.** Jede Exception hat einen klaren Code. “Sonstiger Fehler” ist keine Klasse, sondern eine Bankrotterklärung.
- **Routen.** Jede Klasse hat einen Owner und eine SLA. Niemand wartet, weil “IT vielleicht zuständig” ist.
- **Lernen.** Exception-Daten werden regelmäßig analysiert. Wiederkehrende Ausnahmen werden entweder im Bot behandelt oder am Quellprozess gefixt.

7.3 Senior-Reflexionsfragen

- Wo entsteht der größte Schaden bei Fehlautomation – finanziell, Compliance, Reputation? Welcher Guardrail verhindert ihn zuverlässig?
- Welche Entscheidung ist irreversibel oder teuer (Zugriffskonzept, Orchestrierung, Betriebsmodell)? Wer darf sie treffen?
- Was muss als Guardrail in *jedem* Bot-Design stehen, ohne Ausnahme? (Logging-Minimum, Plausibilitätsgrenzen, Stop-Conditions.)

8. Management-Perspektive: RPA als Portfolio

Auf Wissens-Ebene ist RPA ein Werkzeug. Auf Anwendungs-Ebene ist es eine Sammlung von Bots. Auf Management-Ebene ist es ein *Portfolio mit Lifecycle*.

8.1 Automation Funnel und Portfolio-Steuerung

Ein RPA-Portfolio braucht Kriterien zur Steuerung – nicht nach Bauchgefühl, sondern nach Scoring:

- **Value.** Zeitgewinn, Fehlerreduktion, SLA-Stabilität. Quantifiziert pro Use Case.
- **Risk.** Schadenspotential bei Fehlautomation, Compliance-Relevanz, Reputations-Risiko.
- **Effort.** Bauaufwand, laufende Wartung, Schulungsbedarf.

Ein Use Case wird priorisiert, wenn er einen hohen Wert bei akzeptablem Risiko und ver-

trebarem Aufwand verspricht. Use Cases mit hohem Risiko und niedrigem Wert sind keine Kandidaten – auch wenn sie “automatisierbar” wären.

8.2 Kill Criteria

Wer ein Portfolio steuert, muss auch Bots *abschalten* können. Drei typische Kill Criteria:

- **Stabilität.** Bot fällt häufiger aus, als die manuelle Bearbeitung Zeit gespart hat.
- **Compliance.** Audit-Findings, die nicht in der Zeit der laufenden Wartung behebbar sind.
- **Strategischer Wechsel.** Das Zielsystem wird abgelöst – Bot retten oder Bot retieren?

8.3 Governance und Rollen

Eine pragmatische Rollenverteilung für ein wachsendes RPA-Programm: (vgl. PMI, 2021)

- **Bot Owner.** Verantwortet den einzelnen Bot in Build und Run. Fachlich und technisch zuständig.
- **Process Owner.** Fachlicher Eigentümer des Prozesses, in dem der Bot arbeitet. Definiert Anforderungen, akzeptiert Outputs.
- **Security Owner.** Definiert Baselines (Credentials, Vault, SoD, Logging) und prüft Ausnahmen.
- **Operations / Run.** Betreibt die Plattform (Orchestrator, Server, Updates), reagiert auf Incidents.
- **Change Advisory (light).** Kleines Gremium, das nur teure und sensible Bot-Änderungen reviewt.

8.4 Top-Entscheidungen eines RPA-Operating-Modells

1. Welche 5 Use Cases werden zuerst angegangen?
2. Wo ist die Unattended-Grenze (welche Eignungs-Schwellen, welche Plattform-Reife)?
3. Welche Security-Baselines sind nicht verhandelbar?
4. Wie wird das Audit-Trail-Format vereinheitlicht?
5. Wie wird die Ausnahmebehandlung organisiert (CoE oder lokal)?
6. Wer entscheidet über das Retirement eines Bots?

Schluss-Merksatz für Tag 11

RPA wird wirksam, wenn die Organisation gleichzeitig *Bot-Disziplin* und *Portfolio-Steuerung* beherrscht. Ein einzelner Bot ist ein technisches Detail. Ein Portfolio von hundert Bots ist eine **Verantwortungsarchitektur** – und braucht Governance, die nicht bremst, sondern befähigt.

9. Take-aways aus Tag 11

1. **RPA-Eignung in vier Kriterien:** Regelbasiert, Stabil, Volumen, niedrige Ausnahmen.
2. **Plattform-Architektur:** Studio (bauen), Orchestrator (steuern), Run-Umgebung (ausführen). Toolneutrale Begriffe.
3. **Attended startet schneller, unattended skaliert besser.** Eine Stufenstrategie ist meist die Antwort.

4. **Risiken sind asymmetrisch:** ein Bot, einmal falsch gebaut, macht denselben Fehler tausendmal.
5. **Ausnahmebehandlung entscheidet ROI.** Klassifizieren, routen, lernen.
6. **Audit Trail ist Pflicht,** nicht Kür. Wer ohne baut, hat keine Compliance-Antwort.
7. **Entscheidungen unter Unsicherheit:** A-V-F-K – Annahmen, Verworfen Alternativen, Frühwarnsignale, Kill Criteria.
8. **RPA ist ein Portfolio,** kein einzelner Bot. Steuerung mit Value/Risk/Effort, klaren Kill Criteria, lean Governance.

10. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018).

Fundamentals of business process management (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>

Kahneman, D. (2011).

Thinking, fast and slow. Farrar, Straus and Giroux.

Lacity, M., & Willcocks, L. P. (2018).

Robotic process and cognitive automation: The next phase. SB Publishing.

Project Management Institute. (2021).

A guide to the Project Management Body of Knowledge (PMBOK® Guide) (7th ed.). Project Management Institute.

Scheppler, B., & Weber, C. (2020).

Robotic Process Automation. *Informatik-Spektrum*, 43(2), 152–156. <https://doi.org/10.1007/s00287-020-01263-6>

Smeets, M., Erhard, R., & Kaußler, T. (2019).

Robotic Process Automation (RPA) in der Finanzwirtschaft: Technologie – Implementierung – Erfolgsfaktoren. Springer Gabler. <https://doi.org/10.1007/978-3-658-26564-2>

Syed, R., Suriadi, S., Adams, M., Bandara, W., Leemans, S. J. J., Ouyang, C., ter Hofstede, A. H. M., van de Weerd, I., Wynn, M. T., & Reijers, H. A. (2020).

Robotic process automation: Contemporary themes and challenges. *Computers in Industry*, 115, 103162. <https://doi.org/10.1016/j.compind.2019.103162>

van der Aalst, W. M. P., Bichler, M., & Heinzl, A. (2018).

Robotic process automation. *Business & Information Systems Engineering*, 60(4), 269–272. <https://doi.org/10.1007/s12599-018-0542-4>

Willcocks, L. P., Lacity, M., & Craig, A. (2017).

Robotic process automation: Strategic transformation lever for global business services? *Journal of Information Technology Teaching Cases*, 7(1), 17–28. <https://doi.org/10.1057/s41266-016-0016-9>

11. Glossar

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

Attended Bot

RPA-Bot, der am Arbeitsplatz eines Mitarbeitenden läuft; vom Mitarbeitenden getriggert und überwacht. Geeignet für Assistenz und Teilschritte.

Audit Trail

Lückenlose Aufzeichnung aller Bot-Aktionen mit Bot-ID, Version, Quelle, Aktion, Ergebnis, Bearbeiter und Begründung. Pflicht für regulierte Prozesse.

Automation Funnel

Trichterförmiger Auswahlprozess: Ideen → Eignungs-Check → PoC → Pilot → Betrieb. Filtert ungeeignete Use Cases früh aus.

Bot Owner

Verantwortlich für einen einzelnen Bot in Build und Run. Fachlich und technisch zuständig.

Citizen Developer

Fachbereichsperson, die mit Low-Code-Werkzeugen Automation baut. Effizient mit Standards, gefährlich ohne.

Credential Vault

Zentrales System zur sicheren Verwaltung von Zugangsdaten. Bots greifen über Tokens darauf zu, keine Passwörter im Code.

Exception Queue

Warteschlange für Vorgänge, die der Bot nicht verarbeiten konnte. Wird nach Code/Owner/SLA bearbeitet.

Idempotenz

Eigenschaft: mehrfache Ausführung erzeugt dasselbe Ergebnis wie eine einzige. In RPA wichtig bei Retries und Wiederanlaufen.

Kill Criteria

Vorab definierte Schwellen, ab denen ein Bot abgeschaltet wird (z. B. Exception Rate, Fehlbuchungsquote).

Least Privilege

Sicherheitsprinzip: Ein Bot bekommt nur die Rechte, die er für seine Aufgabe braucht – nicht mehr.

Orchestrator

Zentrales Steuerungs- und Monitoring-System einer RPA-Plattform. Verwaltet Bots, Queues, Credentials, Logs.

Process Owner

Fachlicher Eigentümer des Prozesses, in dem der Bot arbeitet. Definiert Anforderungen, akzeptiert Outputs.

RPA

Robotic Process Automation. Software, die UI-basierte oder schnittstellenbasierte Aufgaben wie ein menschlicher Anwender ausführt. (van der Aalst et al., 2018)

Run-Umgebung

Ausführungsknoten eines Bots – Arbeitsplatz (attended) oder Server/VM (unattended). Mit Iso-

lation, Ressourcen, Verfügbarkeit.

Schatten-IT

IT-Lösungen, die ohne offizielle Freigabe oder Inventur in Fachbereichen entstehen. Häufiges Risiko bei unkontrollierter RPA-Einführung.

Segregation of Duties (SoD)

Regel: kritische Aktionen werden nicht von einer Person (oder einem Bot) allein ausgeführt. “Anlegen” und “Freigeben” getrennt.

Studio / Designer

Werkzeug zum Bauen von Bots. Enthält Workflows, Activities, Variablen, Bedingungen.

Unattended Bot

RPA-Bot, der serverseitig vollautomatisch läuft, ohne menschliche Anwesenheit. Höhere Skalierung, höhere Governance-Anforderungen.

Workflow (in RPA)

Zusammenhängende Folge von Activities innerhalb eines Bots. Bausteine: Sequenzen, Verzweigungen, Schleifen.